TN0474

ne-eeg-server: A Public MCP Server for EEG Analysis with Neuroelectrics Data Formats

Authors	G. Ruffini, K (Neuroelectrics)
Date	April 08, 2026
Version	0.1.0
Status	Draft
Repository	github.com/Neuroelectrics/ne-eeg-server
License	Apache 2.0

1. Introduction

This Technical Note describes **ne-eeg-server**, a new open-source Model Context Protocol (MCP) server that exposes EEG analysis tools for Neuroelectrics data formats. The server is designed for public release and operates exclusively on local EEG files with no cloud connectivity, no patient data management, and no stimulation functionality.

The project is derived from the internal MCP server (**neuroelectrics-mcp-demo**), which provides a broader set of tools including stimulation session programming, patient management, device fleet status, foundation model embeddings, and EEG analysis. The public server extracts and refines the EEG analysis pipeline while deliberately excluding all proprietary stimulation IP.

2. Motivation

MCP is emerging as the industry standard protocol for exposing tool capabilities to large language models. As of early 2026, MCP is supported by Claude (Anthropic), ChatGPT (OpenAI), Gemini (Google), VS Code / GitHub Copilot (Microsoft), Cursor, Windsurf, and dozens of other clients. A

public EEG analysis server enables researchers and clinicians to interact with Neuroelectrics EEG data via natural-language interfaces without requiring programming expertise.

Separating the EEG analysis tools from the internal server addresses two key concerns: (1) the internal server includes parsing and validation of the proprietary `.neprot` format, which must remain confidential; and (2) a focused EEG-only tool set provides a cleaner, more auditable codebase for open-source release under the Apache 2.0 license.

3. Scope and Boundaries

3.1 In scope

Format	Extension	Description
.easy	.easy, .easy.gz	Native NE ASCII format. Raw EEG + markers in nV, tab-separated.
.nedf	.nedf	NE extension of EDF. ASCII XML header + 24-bit binary EEG data.
.edf	.edf	Standard European Data Format. 16-bit, DC-filtered, widely compatible.

3.2 Explicitly out of scope

Excluded capability	Reason
Stimulation protocol tools	Proprietary .neprot format; safety-critical stimulation parameters
Patient data management	Cloud/platform dependency; GDPR / data privacy scope
Device fleet management	Requires authenticated NE cloud connection
Foundation model embeddings	Proprietary NE model weights; cloud API
Biomarker computation	Platform-level feature, not local file analysis

4. Architecture

The server follows a layered architecture with clear separation between protocol transport, tool dispatch, format readers, analysis pipeline, and report generation.

4.1 Analysis pipeline

The signal processing pipeline applies: (1) load and normalize raw data to microvolts; (2) 1-40 Hz bandpass filter (4th-order Butterworth); (3) 50/60 Hz notch filter; (4) PSD estimation via Welch method; (5) band power computation for delta (0.5-4 Hz), theta (4-8 Hz), alpha (8-13 Hz), beta (13-30 Hz), and gamma (30-125 Hz); (6) per-channel QC metrics including RMS, kurtosis, event rate, line noise power, and suspicious PSD peak detection; (7) PASS/FAIL classification per channel.

4.2 Transport modes

The server supports two transport modes: **stdio** (standard input/output, used by Claude Desktop, Claude Code, Gemini CLI, Cursor, and other local MCP clients) and **SSE** (Server-Sent Events over HTTP, used by ChatGPT and remote clients). The transport is selected at startup via the `--transport` flag.

5. MCP Tool Reference

#	Tool	Description
1	file_info	Extract metadata: format, channels, sample rate, duration, device class, file size.
2	list_events	Extract all markers/events with timestamps from a recording.
3	signal_quality	Per-channel QC metrics (RMS, kurtosis, event rate, line power) with PASS/FAIL. Returns JSON.
4	analyze_eeg	Full spectral analysis: PSD, band powers, alpha/theta ratio, peak alpha frequency, frontal alpha asymmetry. Returns JSON + inline PSD plot.
5	generate_qc_report	Generates a branded PDF Signal Quality report with per-channel metrics table, PSD plots, and time-series.
6	generate_analysis_report	Generates a branded PDF EEG Analysis report with raw traces, band power heatmap, alpha reactivity, spectral summary.

6. Migration from Internal Server

The codebase is derived from **neuroelectrics-mcp-demo** using a copy-and-strip strategy. A mandatory security audit uses grep-based scanning for keywords including `stim`, `protocol`, `patient`, `neprot`, `current_ua`, and `sham` to verify no proprietary content leaks into the public repository.

7. Installation and Usage

7.1 Install from PyPI (planned)

```
pip install ne-eeg-server
```

7.2 Claude Desktop

Add the following to `claude_desktop_config.json` (located at `~/Library/Application Support/Claude/` on macOS or `%APPDATA%\Claude\` on Windows):

```
{
  "mcpServers": {
    "ne-eeg-server": {
      "command": "ne-eeg-server",
      "args": []
    }
  }
}
```

Restart Claude Desktop. The server tools will appear automatically when Claude detects EEG-related queries.

7.3 Claude Code

```
claude mcp add ne-eeg-server -- ne-eeg-server
```

Verify with `/mcp` inside Claude Code.

7.4 Gemini CLI

Gemini CLI supports MCP servers via stdio transport. Add the following to `~/.gemini/settings.json`:

```
{
  "mcpServers": {
    "ne-eeg-server": {
      "command": "ne-eeg-server",
      "args": []
    }
  }
}
```

Alternatively: `gemini mcp add ne-eeg-server -- ne-eeg-server`

Verify with `/mcp` inside Gemini CLI. Note: Gemini CLI currently supports MCP tools only (not resources or prompts).

7.5 Gemini API (Python SDK)

The Gemini Python SDK supports MCP servers programmatically (experimental). Install the ne-eeg-server package and the Gemini SDK (`pip install google-genai`):

```
from fastmcp import Client
from google import genai
import asyncio

mcp_client = Client("ne-eeg-server")
gemini_client = genai.Client()

async def main():
  async with mcp_client:
    response = await gemini_client.aio.models.generate_content(
      model="gemini-2.5-flash",
      contents="Analyze /path/to/recording.easy",
      config=genai.types.GenerateContentConfig(
        tools=[mcp_client.session],
      ),
    )
    print(response.text)

asyncio.run(main())
```

7.6 ChatGPT (Developer Mode)

ChatGPT supports MCP servers via remote HTTP connections. This requires: (1) a ChatGPT Pro, Team, Enterprise, or Edu subscription; (2) Developer Mode enabled in ChatGPT settings; (3) the ne-eeg-server running with SSE transport on a publicly accessible host.

Step 1 - Start the server with SSE transport:

```
ne-eeg-server --transport sse --port 8080
```

The server must be reachable from the internet (e.g., via a cloud VM, ngrok tunnel, or Docker MCP Toolkit). For local testing: `ngrok http 8080`

Step 2 - Add the server in ChatGPT: In ChatGPT, open the app/connector settings and add a new MCP server with the public URL (e.g., `https://your-host.ngrok.io/sse`). The server tools will appear in Developer Mode.

Important limitations: ChatGPT only supports remote (HTTP/SSE) MCP servers, not stdio. Because ne-eeg-server operates on local files, the EEG files must be accessible to the machine running the server. This makes ChatGPT integration most practical for server-deployed scenarios where EEG data is co-located with the MCP server.

7.7 Other MCP clients

The ne-eeg-server follows the standard MCP specification and should work with any MCP-compatible client that supports stdio transport, including Cursor, Windsurf (Codeium), VS Code with GitHub Copilot, and Firebase Studio. Configuration follows the same JSON pattern as

Claude Desktop.

8. Dependencies

Package	Version	Purpose
mcp	>=1.0.0	MCP SDK
numpy	any	Numerical arrays
scipy	any	Signal processing
matplotlib	any	Plot generation
reportlab	any	PDF report creation
pyedflib	any	EDF/NEDF reading
pydantic	>=2.0	Data validation

9. Future Work

Planned enhancements include: (a) format conversion tools between .easy, .nedf, and .edf; (b) time-segment extraction and export; (c) batch processing across multiple recordings; (d) connectivity and complexity biomarkers (local computation); (e) MNE-Python integration; (f) interactive visualization artifacts for Claude Desktop; and (g) Streamable HTTP transport to replace SSE, following the latest MCP specification updates.

10. References

[1] Model Context Protocol specification: https://modelcontextprotocol.io
[2] NEPy toolbox: https://github.com/Neuroelectrics/NEPy
[3] NIC2 data formats: https://supportne.freshdesk.com/support/solutions/articles/35000294937
[4] NE wiki: https://www.neuroelectrics.com/wiki
[5] Gemini CLI MCP docs: https://geminicli.com/docs/tools/mcp-server/
[6] OpenAI MCP docs: https://developers.openai.com/api/docs/mcp
[7] ChatGPT Developer Mode: https://platform.openai.com/docs/guides/developer-mode